

## Assignment - 8

Q1] What is meant by a virtual address?

- Address generated by CPU during a program's execution.
- It does not correspond to a physical address in memory but instead maps to it via the memory management unit (MMU).
- This abstraction allows processes to operate in isolated memory spaces.

Q2] What is the use of & (address of) operator?

& → Used for any type of datatype.

& = "Address of operator" (in C & C++)

Use:

1] Used to fetch address of variable.

2] Address generated by & operator is 'virtual address' allocated for that variable.

- 3) What is address space of a process
- Refers to the range of memory addresses that the process can use
  - It is a private space that can't be accessed by other processes, unless it's shared
  - Set of logical addresses that a process references in its code.

Ex: 32-bit address allowed, address can range from 0 - 0x7fff ffff that is  $2^{31}$  possible numbers.

- It includes.
  - 1) Code (text) segment
  - 2) Data segment (initialized & uninitialized variables)
  - 3) Stack
  - 4) Heap

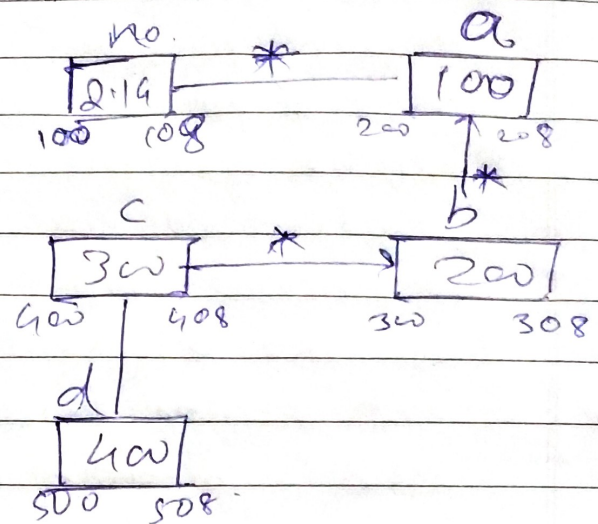
4) Predict the output of below code

- 1) `arr[0] = 10`
- 2) `arr[10] = arr[2] = 30`
- 3) `arr[3-2] = arr[1] = 20`
- 4) `*arr = arr[0] = 100`
- 5) `*arr + 1 = 104`
- 6) `* (arr + 3)`
- 7) `arr[3] =`  
`arr[4] =`  
`arr[5] = 0`  
`2[arr] = arr[2] = 30`

Q7 Predict the output of below code & draw its diagrammatic representation.

→

$\&no = 100$   
 $a = 100$   
 $\&c = 400$   
 $\&d = 500$   
 $d = 400$   
 $**d = 200$   
 $**c = 100$   
 $*b = 100$



Q7 Predict output & draw diagrammatic representation

Size of (no) = 8  
 Size of (a) = 4  
 Size of (b) = 4  
 Size of (c) = 4  
 Size of (d) = 4  
 Size of (\*\*d) = 4  
 Size of (\*\*c) = 4  
 Size of (\*\*\*d) = 8  
 Size of (\*\*\*c) = 8

Q7 What is pointer in C programming? Why to use it?

- Derived Data type in C & C++
- No concept of pointer in Java (reference)
- Defn - variable which stores the memory addresses
  - ⇒ Special variable which can store the memory address of anything.
- Power of pointer - to fetch the data whose address is stored in it.
- Create pointer - points to any primitive data type.

Special variable which holds address, each address is of type - 'Unsigned Long'.

∴ Size of every pointer = 8 byte

Eg: `int *q = &i;`

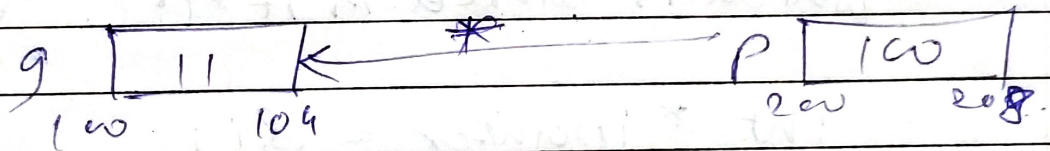
`q` is a pointer which points the integer data type currently it holds the address of `i`.

- `*` ⇒ Reference Operator.

Q1 What are different usage of pointer, explain with example.

→ There are mainly 2 functions of a pointer.  
1) Fetch the value of the variable it is pointing to.  
2) Storing address of that variable.

Ex: `int *q = &i;`  
`*p = &q`



Q2 Dynamic memory allocation

- Pointers are used to allocate memory dynamically using the `calloc()` and `malloc()` functions.

Q3 Passing arguments by reference:

Q4 Pointer Arithmetic - Min 2 pointers of same datatype needed to carry on arithmetical functions (+, -, \*, /)

Q5 This `*` → 1<sup>st</sup> Implicit Hidden parameter  
To call any non-static function we need object of that class, we call any non-static member, that address gets stored inside this `*`

Q7) Write a program to demonstrate different types of pointer which are pointing to primitive datatype.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char ch = 'A';
```

```
    int i = 100;
```

```
    float f = 55.5f;
```

```
    double d = 11.1111;
```

```
    char *chpointer = &ch
```

```
    printf("address of ch = %u \n", &ch;
```

```
    printf("chpointer holds address of ch &  
address is %u \n", chpointer);
```

```
    printf("chpointer the value from address  
which is stored in it of %c \n", *chpointer);
```

```
    int *ipointer = &i;
```

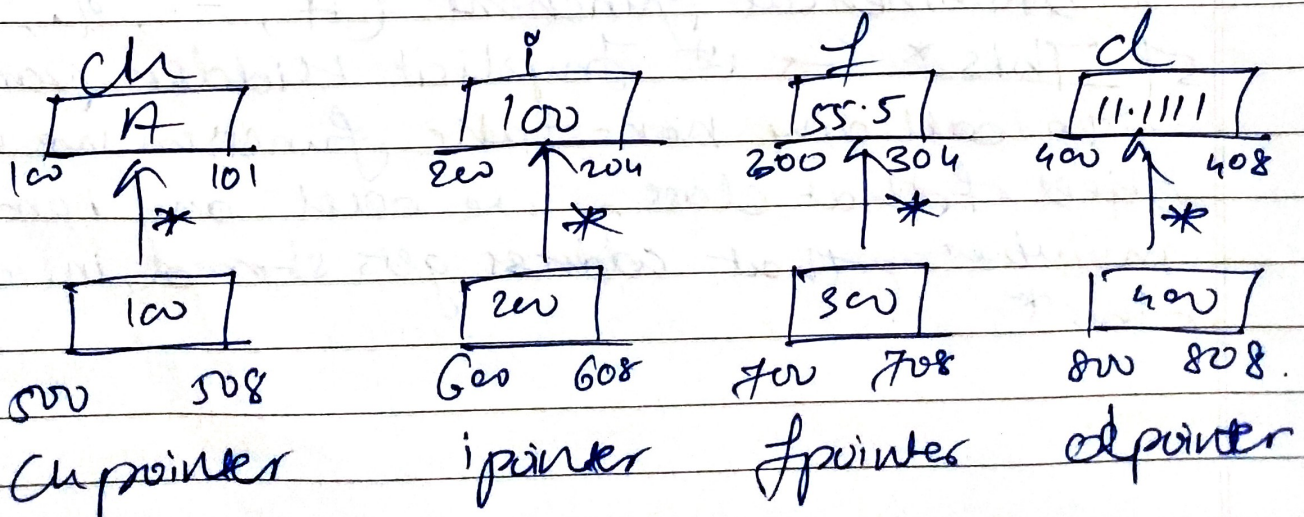
```
    float *fpointer = &f;
```

```
    double *dpointer = &d;
```

```
    printf("size of pointer of %d \n", sizeof(*ipointer));
```

```
    return 0;
```

```
}
```



10/ Read the below statements & write reading statements for some

`int no = 10;`  
`no` is a variable of integer datatype, containing value 10.

a) `int *p = &no;`  
`p` is pointer, pointing to variable `no`, ~~and~~ is of integer datatype.

b) `char ch = 'A';`  
`Char *chptr = &ch`  
- `ch` is a variable of char datatype, holding value 'A'.  
- `chptr` is a pointer, pointing to char datatype and holds the address of `ch`.

c) `double d = 10202020;`  
`double *dbptr = &d;`  
- `d` is a variable of double datatype, initialized with value 10202020.  
- `dbptr` is a pointer, pointing to double datatype and holds address of `d`.